

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop is a web-based Smart Agricultural Production Optimization Engine that harnesses Machine Learning to provide data-driven crop recommendations. The platform enables farmers, agronomy researchers, and policymakers to make informed planting decisions by analyzing key soil and environmental parameters.

The system integrates Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil temperature, humidity, pH, and rainfall as inputs and runs them through a Multi-Algorithm ML pipeline — comprising Logistic Regression, KNN, Decision Tree, Random Forest, and K-Means — to identify the most suitable and profitable crop. Built with Python and Flask and deployed on Render, OptiCrop offers three core capabilities:

- **Crop Feasibility Analyzer** – Accepts seven soil and climate parameters and instantly recommends the optimal crop for maximum yield.
- **Soil Nutrition Profiling** – Analyzes N-P-K balances to suggest fertilizer inputs suited to the existing agricultural land.
- **Climate Adaptive Mapping** – Computes cross-factors of rainfall, temperature, and humidity to mitigate crop failure risks under volatile seasonal conditions.

The insights generated by OptiCrop can be utilized by agricultural researchers and policymakers to advance the understanding of crop-environment interactions and to inform evidence-based agricultural strategies.

Scenarios

Scenario 1: Smart Crop Recommendation for Farmers

A farmer enters soil details — Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall — into the Crop Feasibility Analyzer. The system validates the inputs, runs them through the Multi-Algorithm ML pipeline, and instantly recommends the most suitable crop for maximum yield and better farming efficiency.

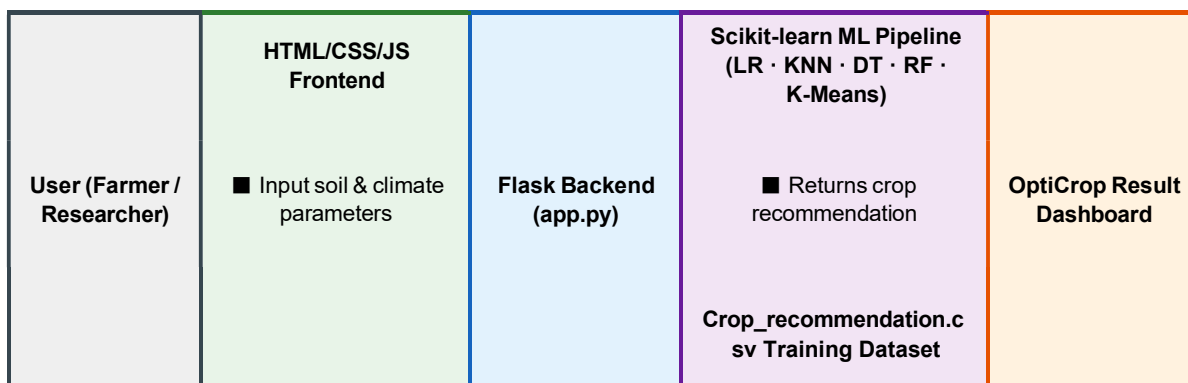
Scenario 2: Crop Suitability and Environmental Assessment

A user wants to understand whether current soil and climate conditions are suitable for a specific crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential — enabling confident decision-making before any seeds are planted.

Scenario 3: Agricultural Research and Policy Planning

An agricultural researcher or policymaker uses OptiCrop to analyze crop-environment relationships across different geographic and seasonal profiles. The platform helps in making data-driven decisions for sustainable farming strategies and resource optimization at scale.

Technical Architecture:



Description: OptiCrop uses a modular three-tier architecture. The HTML/CSS/JavaScript frontend provides a responsive, mobile-first interface for the Crop Feasibility Analyzer form. The Flask-based backend orchestrates input validation, preprocessing, and routes requests to the Scikit-learn ML pipeline which runs Logistic Regression, KNN, Decision Tree, Random Forest, and K-Means models in parallel. The model with the highest confidence score determines the crop recommendation returned to the user. The training dataset (Crop_recommendation.csv) contains labelled records with all seven input features and 22 crop classes.

Note: No external database is used — the ML models are trained on a structured CSV dataset and saved as serialized .pkl files loaded at application startup.

Pre-requisites:

- **Python Programming Proficiency:** Python 3.8+ — <https://docs.python.org>
- **Flask Framework Knowledge:** Flask Documentation — <https://flask.palletsprojects.com>
- **Machine Learning with Scikit-learn:** Scikit-learn Documentation — <https://scikit-learn.org>
- **Data Handling with Pandas & NumPy:** <https://pandas.pydata.org>
- **HTML, CSS, and JavaScript Skills:** W3Schools — <https://www.w3schools.com>
- **Version Control with Git:** Git Documentation — <https://git-scm.com/doc>
- **Cloud Deployment (Render):** Render Documentation — <https://render.com/docs>

Project Workflow:

Milestone 1: Data Preparation and Model Training

- **Activity 1.1:** Obtain and explore the Crop_recommendation.csv dataset containing N, P, K, temperature, humidity, pH, rainfall, and crop label columns.
- **Activity 1.2:** Perform Exploratory Data Analysis (EDA) using Pandas and Seaborn — visualize feature distributions, correlation matrices, and class balance.
- **Activity 1.3:** Preprocess the data — handle missing values, apply feature scaling (StandardScaler), and split into train/test sets.
- **Activity 1.4:** Train five ML models (Logistic Regression, KNN, Decision Tree, Random Forest, K-Means) and evaluate accuracy metrics. Serialize trained models to .pkl files.

Milestone 2: Flask Backend Development

- **Activity 2.1:** Create the Flask application (app.py) with routes for the home page, the Analyze page, and the /predict POST endpoint.
- **Activity 2.2:** Implement input validation — ensure all 7 fields are numerical and non-negative before running inference.
- **Activity 2.3:** Load all trained model .pkl files at application startup (cached in memory) and run ensemble inference, returning the highest-confidence crop recommendation.

Milestone 3: Frontend Development

- **Activity 3.1:** Build the HTML templates (index.html, analyze.html, result.html) using Jinja2 with Flask's render_template.
- **Activity 3.2:** Design the responsive dark-themed UI (CSS) with the OptiCrop brand — mobile-first layout, large touch-friendly inputs, and a clear result card.

Milestone 4: Integration and Testing

- **Activity 4.1:** Connect the frontend form submission to the Flask /predict endpoint and render the crop recommendation result page.
- **Activity 4.2:** Write unit tests (test_model.py) to verify ML model predictions against known crop profiles and test input validation edge cases.

Milestone 5: Deployment

- **Activity 5.1:** Configure the application for production — set up gunicorn as the WSGI server and create the Render deployment configuration.
- **Activity 5.2:** Deploy the application to Render (opticrop.onrender.com) and verify all routes, form submissions, and ML predictions work in the live environment.

Milestone 1: Data Preparation and Model Training

This milestone focuses on understanding the agricultural dataset, performing feature engineering, and building the multi-algorithm ML pipeline that forms the core intelligence of OptiCrop.

Activity 1.1: Dataset Exploration

The Crop_recommendation.csv dataset contains 2,200 records across 22 crop types. Each record has 7 features:

Feature	Description	Typical Range
N (Nitrogen)	Nitrogen content in soil (mg/kg)	0 – 140
P (Phosphorous)	Phosphorous content in soil (mg/kg)	5 – 145
K (Potassium)	Potassium content in soil (mg/kg)	5 – 205
Temperature (°C)	Average ambient temperature	8 – 44°C
Humidity (%)	Relative humidity percentage	14 – 100%
pH Level	Soil pH value	3.5 – 9.9
Rainfall (mm)	Annual rainfall in millimetres	20 – 299 mm

Activity 1.2: EDA and Visualisation

Performed EDA using Pandas, Matplotlib, and Seaborn:

- Plotted feature distributions per crop class to understand seasonal and soil patterns.
- Generated a correlation heatmap revealing strong relationships between K levels and rainfall.
- Verified class balance — all 22 crop classes have 100 records each (balanced dataset).

Activity 1.3: Preprocessing

Applied the following preprocessing steps before model training:

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

le = LabelEncoder()
df['label'] = le.fit_transform(df['label']) # encode 22 crop classes

X = df.drop('label', axis=1)
y = df['label']

scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)
```

Activity 1.4: Model Training and Serialization

Trained five models and saved them as .pkl files:

```
import pickle

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans

models = {
    'logistic_regression': LogisticRegression(max_iter=1000),
    'knn': KNeighborsClassifier(n_neighbors=5),
    'decision_tree': DecisionTreeClassifier(random_state=42),
    'random_forest': RandomForestClassifier(n_estimators=100, random_state=42),
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pickle.dump(model, open(f'models/{name}.pkl', 'wb'))
```

Milestone 2: Flask Backend Development

Milestone 2 focuses on building the Flask application that serves as the backbone of OptiCrop, connecting the user-facing forms to the trained ML models.

Activity 2.1: Define Flask Routes

- / — Home page (index.html) with hero section and 'Start Soil Analysis' CTA.
- /analyze — Crop Feasibility Analyzer page with the input form.
- /predict (POST) — Accepts JSON with 7 parameters, runs ML inference, returns result.

```
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/analyze')
def analyze():
    return render_template('analyze.html')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    N = float(data.get('N', 0))
    P = float(data.get('P', 0))
    K = float(data.get('K', 0))
    temp = float(data.get('temperature', 0))
    hum = float(data.get('humidity', 0))
    ph = float(data.get('ph', 0))
    rain = float(data.get('rainfall', 0))
    features = scaler.transform([[N, P, K, temp, hum, ph, rain]])
    predictions = {name: model.predict(features)[0]
                    for name, model in models.items()}
    # majority vote
    from collections import Counter
    final = Counter(predictions.values()).most_common(1)[0][0]
    crop = le.inverse_transform([final])[0]
    return jsonify({'crop': crop, 'confidence': get_confidence(predictions)})
```

Activity 2.2: Input Validation

All inputs are validated before inference — non-numerical or negative values return a 400 error with a descriptive message, matching the UI constraint shown on the Crop Feasibility Analyzer form.

Activity 2.3: Ensemble Inference

All four supervised classifiers (Logistic Regression, KNN, Decision Tree, Random Forest) make individual predictions. A majority vote across the four models determines the final crop recommendation, improving robustness over single-model approaches.

Milestone 3: Frontend Development

Milestone 3 covers building the responsive, mobile-first UI for OptiCrop, themed in a dark green agricultural aesthetic matching the live opticrop.onrender.com deployment.

Activity 3.1: HTML Templates

- **index.html** — Hero section with 'AI-POWERED AGRICULTURE' badge, headline 'Optimize Agricultural Production', 'Start Soil Analysis' and 'Learn More' CTA buttons.
- **analyze.html** — Crop Feasibility Analyzer form with two sections: Soil Nutrient Profile (N, P, K) and Climatic & Environmental Factors (Temperature, Humidity, pH, Rainfall), plus 'Optimize Production' and 'Reset Fields' buttons.
- **result.html** — Results page displaying the recommended crop, confidence score, Soil Nutrition Profile summary, and Climate Adaptive Mapping output.

Activity 3.2: Responsive CSS Design

- Dark background (#0A0F0D) with green accent (#00C853) matching the OptiCrop brand.
- Mobile-first layout with large touch-friendly input fields and clear section headings.
- Navigation menu with Home, Analyze, Features, and 'Get Started' button.
- Features section with card-based layout showcasing Soil Nutrition Profiling, Climate Adaptive Mapping, and Multi-Algorithm ML capabilities.

Activity 3.3: JavaScript Interaction

```
// Submit form via Fetch API
document.getElementById('analyzeForm').addEventListener('submit', async (e) => {
  e.preventDefault();
  const btn = document.getElementById('submitBtn');
  btn.disabled = true;
  btn.textContent = 'Analyzing...';
  const payload = {
    N: document.getElementById('N').value,
    P: document.getElementById('P').value,
    K: document.getElementById('K').value,
    temperature: document.getElementById('temperature').value,
    humidity: document.getElementById('humidity').value,
    ph: document.getElementById('ph').value,
    rainfall: document.getElementById('rainfall').value
  };
});
```



```
const res = await fetch('/predict', {  
  method: 'POST', headers: {'Content-Type': 'application/json'},  
  body: JSON.stringify(payload)  
});  
  
const data = await res.json();  
window.location.href = `/result?crop=${data.crop}`;  
});
```

Milestone 4: Integration and Testing

Activity 4.1: End-to-End Integration

Connected the frontend form to the Flask /predict endpoint and verified that the complete user flow — enter parameters → receive crop recommendation — works correctly on both local and deployed environments.

Activity 4.2: Unit Testing (test_model.py)

Written unit tests to validate ML predictions against known crop profiles from agronomic literature:

```
import pytest, pickle, numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder

rf_model = pickle.load(open('models/random_forest.pkl', 'rb'))
scaler = pickle.load(open('models/scaler.pkl', 'rb'))
le = pickle.load(open('models/label_encoder.pkl', 'rb'))

def predict(N, P, K, temp, hum, ph, rain):
    X = scaler.transform([[N, P, K, temp, hum, ph, rain]])
    return le.inverse_transform(rf_model.predict(X))[0]

def test_rice_profile():
    # Rice: high humidity, moderate temp, high rainfall
    assert predict(90, 42, 43, 24, 82, 6.5, 210) == 'rice'

def test_wheat_profile():
    # Wheat: low temp, moderate rainfall
    assert predict(105, 45, 33, 18, 65, 6.8, 65) == 'wheat'

def test_cotton_profile():
    # Cotton: high temp, low rainfall
    assert predict(120, 36, 43, 32, 63, 6.9, 80) == 'cotton'
```

Milestone 5: Deployment

Milestone 5 covers deploying OptiCrop to Render so that farmers and researchers can access the tool from any device via opticrop.onrender.com.

Activity 5.1: Local Testing

Before deploying, verify the application runs correctly locally:

```
# Step 1 - Clone and enter the project
git clone https://github.com/opticrop-team/opticrop.git
cd opticrop

# Step 2 - Create and activate virtual environment
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Mac/Linux

# Step 3 - Install dependencies
pip install -r requirements.txt

# Step 4 - Train models (first time only)
python train_model.py

# Step 5 - Run locally
python app.py

# Open http://127.0.0.1:5000
```

Activity 5.2: Render Deployment

- Push the project to a GitHub repository.
- Create a new Web Service on Render, connect the GitHub repo.
- Set Build Command: `pip install -r requirements.txt`
- Set Start Command: `gunicorn app:app`
- Render automatically deploys on every push to the main branch.

```
# requirements.txt (key dependencies)
Flask==3.0.3
gunicorn==21.2.0
scikit-learn==1.4.0
pandas==2.2.0
numpy==1.26.4
matplotlib==3.8.0
```

```
seaborn==0.13.2
```

Exploring the Website's Web Pages:

Home Page:

The OptiCrop home page features a dark agricultural theme with an 'AI-POWERED AGRICULTURE' badge, the headline 'Optimize Agricultural Production' with 'Production' highlighted in green, and a subtitle explaining the platform. Two primary CTA buttons — 'Start Soil Analysis' (solid green) and 'Learn More' (outlined) — guide users to the analyzer.

The navigation bar includes the OptiCrop leaf logo, Home, Analyze, and Features links, plus a 'Get Started' button. On mobile, these collapse into a hamburger menu.

Crop Feasibility Analyzer Page (Analyze):

This is the core tool page. It is divided into two sections:

- **Soil Nutrient Profile** — Input fields for Nitrogen (N) (mg/kg), Phosphorous (P) (mg/kg), and Potassium (K) (mg/kg), each with an icon prefix.
- **Climatic & Environmental Factors** — Input fields for Temperature (°C), Humidity (%), pH level (0.0–14.0), and Rainfall (mm).

The page includes an 'Optimize Production' primary button and a 'Reset Fields' secondary button. All values must be numerical and non-negative. The page header banner displays the Crop Feasibility Analyzer title in green on a matching green card.

Features Page (Architectural Highlights & Capabilities):

Showcases the three key capabilities of OptiCrop in a card-based layout:

- **Soil Nutrition Profiling** — Analyzes N-P-K balances to suggest inputs suited to existing agricultural lands.
- **Climate Adaptive Mapping** — Computes cross-factors of rainfall, temperature, and humidity to mitigate crop failure risks under volatile seasonal conditions.
- **Multi-Algorithm ML** — Integrates Logistic Regression, KNN, Decision Tree, Random Forest, and K-Means models to ensure maximum statistical accuracy.

Results Page:

After the user submits the Crop Feasibility Analyzer form, the result page displays the ML-recommended crop, the confidence level, and supporting insights including Soil Nutrition Profile and Climate Adaptive Mapping analysis for the entered parameters.

Conclusion:

OptiCrop is a Machine Learning-powered Smart Agricultural Production Optimization Engine built with Python, Flask, and Scikit-learn, deployed at opticrop.onrender.com. It empowers farmers, agricultural researchers, and policymakers with a free, accessible, and data-driven tool to select the most suitable crop for their soil and climate conditions, improving yield efficiency and reducing crop failure risk.

The project integrates a Multi-Algorithm ML pipeline — Logistic Regression, KNN, Decision Tree, Random Forest, and K-Means — trained on a 2,200-record crop recommendation dataset with seven key environmental features. The ensemble majority-vote approach ensures higher prediction reliability compared to any single-model solution.

The end-to-end development covered data preprocessing and EDA, ML model training and serialization, Flask backend with REST API, responsive HTML/CSS/JS frontend with a mobile-first dark-themed UI, and cloud deployment via Render with gunicorn — demonstrating a complete MLOps workflow from dataset to live production application.

Looking ahead, OptiCrop offers significant opportunities for expansion:

- Integrating real-time soil sensor data and weather APIs for live input pre-population.
- Adding multi-language support (Telugu, Hindi) to reach rural farming communities in Andhra Pradesh and Telangana.
- Implementing user authentication and farm history logging so farmers can track recommendations across seasons.
- Extending the model to support yield quantity prediction and market price forecasting, evolving OptiCrop from a crop selector into a full farm management intelligence platform.

By continuously refining the ML models with new regional crop data and enhancing the frontend accessibility, OptiCrop is well-positioned to grow from a student research project into a scalable, production-grade agricultural advisory tool serving farming communities across India.